Arquitetura de Dados Híbrida para Varejo Omnichannel

Do pedido ao insight analítico — VarejoMix

Projeto técnico acadêmico

PostgreSQL • MongoDB • Python • Parquet • Delta Lake • DuckDB

15 de outubro de 2026

Sumário

1 Contextualização e objetivos 1
2 Requisitos e arquitetura 1
3 Modelagem relacional e integridade 1
4 Eventos MongoDB e decisão de modelagem 2
5 Bronze, Silver, Gold e histórico Delta 2
6 Modelo dimensional e consultas 3
7 Índices, plano e testes 3
8 Python, features e limitações 4
9 Caso público e conclusão 4
10 Glossário 5
11 Referências 5

1 Contextualização e objetivos

A VarejoMix representa um varejista fictício com lojas físicas e comércio eletrônico. O problema é comum em operações omnichannel: pedidos e catálogo são estruturados e sujeitos a regras transacionais, enquanto navegação e pesquisa são fluxos de eventos flexíveis. Relatórios que misturam essas fontes sem chaves e granularidade explícitas podem duplicar vendas ou contar comportamento sem um denominador estável. O objetivo foi construir uma prova acadêmica executável, do registro do pedido à consulta analítica e ao dataset de clientes. A solução usa tecnologias abertas e uma escala intencionalmente pequena. Foram produzidos dados determinísticos, scripts SQL, adaptadores Python, arquivos Parquet, tabela Delta versionada, notebook, testes e documentação. O trabalho não treina modelo de recompra, não representa um sistema de produção e não declara um serviço Mongo como conectado quando a sandbox não pôde inicializá-lo.

2 Requisitos e arquitetura

A fonte transacional no PostgreSQL normaliza clientes, categorias, produtos, lojas, cabeçalhos e itens de pedido, pagamentos e movimentos de estoque. O MongoDB mantém um documento por evento de comportamento. SQLAlchemy e pandas suportam extração; Python tipa, valida e deduplica; Bronze preserva snapshots por hash, Silver armazena registros curados e rejeições, e Gold oferece uma estrela em Parquet. Delta Lake acrescenta o log de commits numa tabela curada. DuckDB consulta os arquivos localmente.
A Figura 1 mostra o fluxo. `--source fixtures` usa CSV/JSON determinísticos para funcionar sem serviços. `--source services` lê PostgreSQL e MongoDB quando as URLs estão configuradas. O caminho de serviços é implementado no código, mas exige infraestrutura ativa. A separação permite reproduzir transformações sem apresentar arquivos locais como uma conexão a banco.

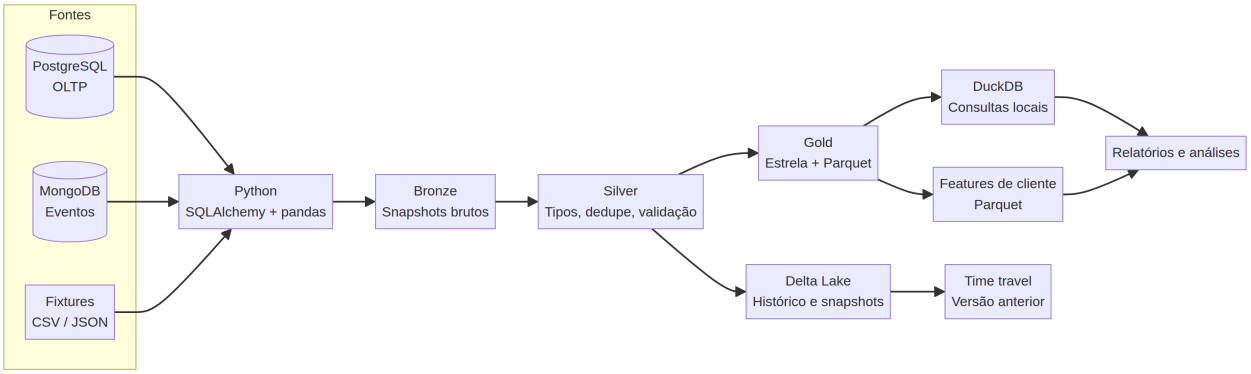


Figura 1: Fluxo local de ingestão, curadoria, histórico Delta e consumo analítico.

As três camadas distinguem preservação e semântica. Bronze registra a entrada identificada pela soma de verificação do conteúdo. Silver conserva proveniência, identificador de execução, timestamp de processamento em UTC, tipos, chaves deduplicadas e validações. Gold exclui pedidos cancelados da fato de venda e expõe dimensões e métricas. Particionamentos por mês de pedido e dia de evento limitam a proliferação de pastas por cliente ou produto.

3 Modelagem relacional e integridade

O processamento transacional on-line (OLTP) mantém cada entidade compartilhada em sua tabela. Produtos referenciam categorias; pedidos referenciam cliente e loja; itens referenciam pedido e produto;

pagamentos referenciam pedidos; movimentos referenciam loja e produto. Chaves primárias e estrangeiras preservam as relações. `UNIQUE` protege SKU, e-mail, pagamento por pedido e a combinação de produto/pedido. `CHECK` controla preços, desconto, quantidade, devolução e estados permitidos. PostgreSQL documenta esses mecanismos como restrições de domínio e integridade; `NOT NULL` permanece necessário porque um `CHECK` que recebe nulo pode não rejeitar a linha [2].

O preço presente no item é histórico e não é substituído pelo preço corrente do catálogo. O total do pedido é armazenado no cabeçalho e reconciliado com a soma dos itens no pipeline e nos testes; uma restrição `CHECK` por linha não calcula agregados de outras linhas. O estoque é derivado como soma das movimentações assinadas. A ruptura operacional é saldo menor ou igual a zero na loja/produto; não se estima disponibilidade futura nem reserva.

O snapshot contém 10 clientes, seis produtos, três unidades (duas lojas e uma unidade virtual), 18 pedidos, 22 itens, 18 pagamentos e 14 movimentos de estoque. Há 17 pedidos concluídos e um cancelado. Um item possui uma unidade devolvida, permitindo demonstrar o abatimento proporcional de desconto na devolução. Os scripts aceitam inserção por chaves declaradas e são reexecutáveis por `ON CONFLICT`.

4 Eventos MongoDB e decisão de modelagem

O documento de comportamento tem `event_id`, cliente opcional, sessão, instante ISO-8601 UTC, canal, tipo de evento e `properties`. O banco de documentos (NoSQL) é adequado a campos cujo conjunto muda entre `search`, `add_to_cart` e `purchase`. A regra de desenho adotada segue o padrão de acesso: informações pequenas que são lidas juntas ficam embutidas, e entidades compartilhadas ou de crescimento independente são referenciadas por IDs. O guia do MongoDB recomenda modelar conforme a forma de leitura; embedding pode reduzir joins, enquanto referências evitam duplicação e crescimento excessivo [3].

Uma sessão contém muitos eventos e um cliente pode aparecer em diversas sessões; sessões anônimas não requerem `customer_id`. `session_id` e `customer_id` são referências lógicas e não oferecem consistência distribuída com o PostgreSQL. Eventos são tratados como append-only e `event_id` único; os índices previstos cobrem ID, sessão+tempo, tipo+tempo e cliente+tempo. O JSON Schema valida campos e formato. `python/seed_mongo.py` instala validador e índices e faz upsert. Na execução documentada, não havia Docker/Compose nem serviço Mongo acessível na sandbox; essa conexão permanece dependente de execução local pelo Compose.

5 Bronze, Silver, Gold e histórico Delta

Os CSV/JSON sintéticos são gravados em `data/bronze/run_id=<hash>`. O hash de entrada mantém o mesmo caminho ao repetir o conteúdo. A camada Silver deduplica IDs naturais e rejeita, por exemplo, horários não parseáveis ou itens com pedido/produto ausente; o snapshot correto observou zero rejeições. Gold tem 21 linhas para 21 `order_item_id` elegíveis, uma linha por item concluído. `fact_sales.parquet` dá acesso direto, enquanto a cópia por mês apresenta o layout particionado. Dimensões e a tabela de resumo gerada pelo DuckDB também ficam disponíveis em Parquet/CSV.

O formato transacional de tabela sobre Parquet (Delta Lake) foi gravado por delta-rs sobre Parquet com particionamento mensal. A primeira gravação produziu versão 0; repetir o mesmo snapshot substituiu os dados da tabela em vez de acrescentar cópias e criou versão 1. A evidência lê versão 0 e compara 21 linhas. Esse comportamento demonstra snapshot histórico, não retenção ilimitada: o time travel requer

a preservação do log e dos arquivos de dados correspondentes [4]. A tabela Delta é distinta de Parquet isolado; seu `_delta_log/` contém commits. A opção delta-rs reduz a complexidade em comparação com exigir Spark para este volume de demonstração.

6 Modelo dimensional e consultas

A fato `dw.fact_sales` tem granularidade declarada de uma linha por item. As dimensões são cliente, produto, data, loja e canal. A Figura 2 apresenta as chaves. O identificador operacional `order_item_id` é único, e testes conferem a unicidade antes e depois de joins. Kimball recomenda declarar o grão em termos de negócio antes de escolher dimensões e fatos; esta implementação segue essa decisão para evitar atribuir o total do pedido a cada produto [8].

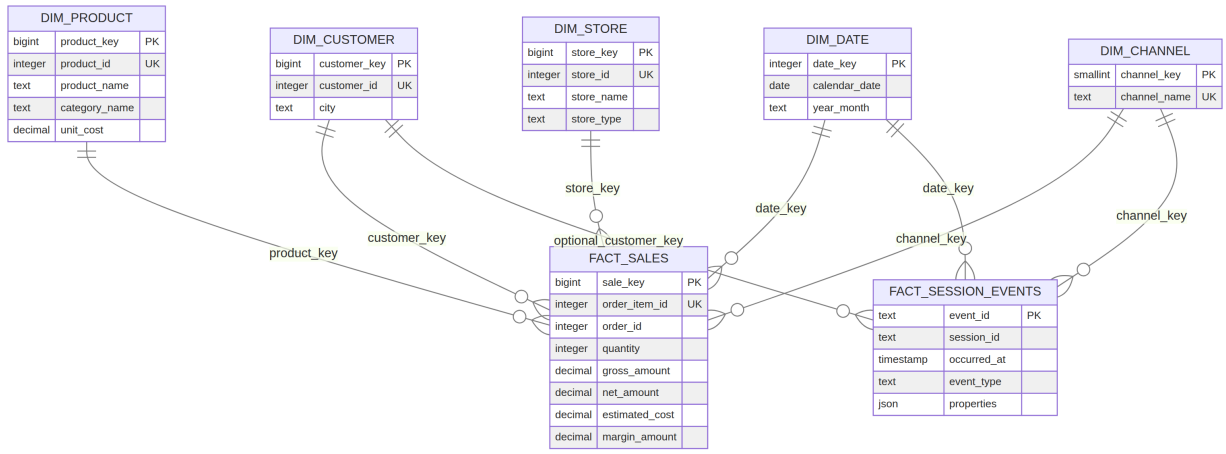


Figura 2: Esquema estrela de vendas e fatos de eventos usados no espelho analítico.

A receita bruta sintética observada foi R$ 4.671,47. A receita líquida foi R$ 4.402,47, calculada como quantidade×preço histórico menos descontos e devoluções líquidas. Pedidos cancelados são excluídos. A margem estimada é receita líquida menos custo unitário das unidades retidas, ou R$ 1.482,47. Não há impostos, frete, taxas, despesas de operação ou política contábil de estoque; portanto, a margem não é medida contábil.

Consultas implementadas cobrem volume e receita por produto, ruptura por saldo agregado, funil comportamental por sessões distintas, coorte pelo mês da primeira compra, recorrência com `LAG`, comparação por canal/loja/período e uma view mensal. A janela do funil é 25/03/2025 a 01/04/2025 inclusive, em UTC. Sua base são oito sessões com `page_view`; quatro também contêm `purchase`, para conversão observada de 50%. Cada estágio conta sessões com pelo menos um evento do tipo; não é atribuição causal nem medição de exposição a anúncios.

7 Índices, plano e testes

Os índices abrangem data/canal, cliente/data, produto/data, loja/produto/tempo do estoque e estado de pagamento. Restrições `UNIQUE` já criam índices para chaves únicas. O plano real do PostgreSQL 16 foi capturado após `ANALYZE`. Na consulta seletiva de vendas do e-commerce em março, o plano escolheu leituras sequenciais na fato e nas dimensões, seguida de joins hash e ordenação. A fato continha apenas 21 linhas; nesse tamanho, o custo estimado de varrer a tabela é baixo. O tempo observado foi 0,268 ms e não

é um benchmark de escala. A existência dos índices registra a intenção de acesso, mas o plano pequeno não comprova ganho de latência.

A suíte automatizada executada retornou 17 testes aprovados. Ela cobre FKs e valores inválidos, reconciliação do total, schema/deduplicação de eventos, timestamps e referência de cliente, métricas por canal, grão e inflação por joins, repetição do pipeline, diretórios Bronze/Silver/Gold, leitura anterior Delta, exportação Parquet, nulos, unicidade do cliente e corte temporal. Os testes relacionais rápidos usam SQLite em memória; os DDL, seeds, estrela, queries e `EXPLAIN ANALYZE` foram executados separadamente contra PostgreSQL 16. MongoDB live e Compose não foram usados na execução aqui relatada.

8 Python, features e limitações

O notebook utiliza SQLAlchemy e `pandas.read_sql` em PostgreSQL quando `DATABASE_URL` existe. A execução registrada consultou seis combinações mensais/canal e 21 itens no banco. Sem a URL, o notebook demonstra o caminho com SQLite em memória alimentado pelo Gold local. A API do pandas recebe uma conexão SQLAlchemy e retorna um DataFrame [6]; SQLAlchemy configura os engines e conexões empregados para a leitura [7]. O Parquet organiza colunas e row groups, e sua separação de metadados e dados ajuda os leitores a localizar os segmentos necessários [5].

O dataset final contém 10 linhas por 12 colunas. Frequência conta pedidos concluídos; monetário é a soma de receita líquida; ticket médio é calculado após consolidar cada pedido para não ponderar pedidos com mais itens. O canal predominante é definido por contagem de pedidos, com desempate alfabético. O corte é 01/04/2025 e somente observações com datas estritamente anteriores são consideradas. O cliente sem compra mantém frequência e valor zero, enquanto recência, última compra, ticket, canal e tempo desde a primeira compra ficam nulos. Eventos sem cliente não entram em features individuais. O artefato prepara variável construída para análise ou ciência de dados (feature) para análises futuras; não contém alvo futuro, modelo treinado, métricas de validação ou tratamento de vazamento para uma tarefa de previsão ainda não implementada.

O escopo mínimo é deliberado: poucas dezenas de linhas, sem orquestrador distribuído, warehouse remoto, credenciais privadas ou serviço pago. Em compensação, não oferece garantias de produção sobre concorrência simultânea do pipeline Python, CDC, evolução de schema, monitoramento operacional ou política completa de privacidade. A reprodução local não exige serviços para o caminho de fixtures, mas é necessária para validar a integração Mongo e o modo `services`.

9 Caso público e conclusão

A própria Uber descreve o Apache Hudi como componente central do seu data lake e relaciona seu surgimento a dados mutáveis, backfills, atualização de esquema e necessidade de processamento incremental. A publicação relata transações ACID, indexação/upserts e leitura incremental, além de integração com Spark em lote, Parquet e ingestão streaming com Flink. Em 2026, a empresa relata 19.500 datasets Hudi sob sua gestão [1]. Trata-se de relato técnico primário da Uber sobre seus próprios sistemas, sem verificação independente neste projeto.

A VarejoMix não reproduz a escala ou a infraestrutura da Uber. O aprendizado transferível é tratar os dados em camadas, manter histórico de commits, evitar recomputações desnecessárias e alinhar consultas a chaves e padrões de acesso. O resultado experimental aqui é mais modesto: fontes artificiais coerentes, con-

sultas SQL reproduzidas, teste automatizado, prova de versionamento e Parquet de features. PostgreSQL foi executado localmente; a conexão live Mongo permanece proposta/implementada no código, mas ainda depende de um serviço Mongo ativo. Essa distinção preserva a reprodutibilidade e evita superestimar o estado do projeto.

10 Glossário

***Delta Lake* – formato transacional de tabela sobre Parquet**: Registra commits e snapshots, permitindo leitura de versões retidas.

***feature* – variável construída para análise ou ciência de dados**: Atributo de cliente calculado a partir de eventos observáveis até um corte.

***lakehouse* – camada analítica sobre armazenamento aberto**: Organiza dados brutos, curados e analíticos sobre formatos de arquivo.

***NoSQL* – banco de documentos**: Modelo flexível para eventos JSON cuja estrutura varia por tipo.

***OLTP* – processamento transacional on-line**: Carga operacional com transações e integridade referencial.

11 Referências

[1] Uber Engineering. **Apache Hudi™ at Uber: Engineering for Trillion-Record-Scale Data Lake Operations**. Publicação técnica da equipe Uber, 2026. https://www.uber.com/us/en/blog/apache-hudi-at-uber/. Acesso em 15 out. 2026.

[2] PostgreSQL Global Development Group. **Constraints**. PostgreSQL Documentation, versão corrente. https://www.postgresql.org/docs/current/ddl-constraints.html. Acesso em 15 out. 2026.

[3] MongoDB, Inc. **Data Modeling in MongoDB**. MongoDB Manual. https://www.mongodb.com/docs/manual/data-modeling/. Acesso em 15 out. 2026.

[4] Delta Lake Project. **Table batch reads and writes — time travel and data retention**. Documentação oficial Delta Lake. https://docs.delta.io/delta-batch/. Acesso em 15 out. 2026.

[5] Apache Software Foundation. **Apache Parquet File Format**. Documentação oficial do formato. https://parquet.apache.org/docs/file-format/. Acesso em 15 out. 2026.

[6] pandas development team. **pandas.read_sql**. Documentação oficial pandas 3.0.6. https://pandas.pydata.org/docs/reference/api/pandas.read_sql.html. Acesso em 15 out. 2026.

[7] SQLAlchemy authors and contributors. **Engine Configuration**. SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/core/engines.html. Acesso em 15 out. 2026.

[8] Ralph Kimball. **Declaring the Grain**. Kimball Group, 1 mar. 2003. https://www.kimballgroup.com/2003/03/declaring-the-grain/. Acesso em 15 out. 2026.